

Task – 1

```
UW PICO 9.09 File: Practical-4.c Modified
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t child1, child2, child3;
    int status;

    child1 = fork();
    if (child1 == 0)
    {
        printf("Child 1: PID = %d\n", getpid());
        sleep(3);
        printf("Child 1 completed.\n");
        exit(10);
    }

    child2 = fork();
    if (child2 == 0)
    {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        printf("Child 2 completed.\n");
        exit(20);
    }

    child3 = fork();
    if (child3 == 0)
    {
        printf("Child 3: PID = %d\n", getpid());
        sleep(2);
        printf("Child 3 completed.\n");
        exit(30);
    }

    printf("Parent: PID = %d\n", getpid());
    printf("\nUsing wait():\n");

    pid_t finished = wait(&status);

    printf("wait() collected child PID: %d\n", finished);
    printf("Exit status: %d\n", WEXITSTATUS(status));
    printf("\nUsing waitpid() for Child 1:\n");
    waitpid(child1, &status, 0);
    printf("waitpid() collected Child 1 PID: %d\n", child1);
    printf("Exit status: %d\n", WEXITSTATUS(status));
    waitpid(child3, &status, 0);
    printf("waitpid() collected Child 3 PID: %d\n", child3);
    printf("Exit status: %d\n", WEXITSTATUS(status));

    return 0;
}
```

Get Help Exit WriteOut Justify Read File Where is Prev Pg Next Pg Cut Text UNCut Text Cur Pos To Spell

Compilation, Input & Output :

```
boddanapudiachandrarao@boddanaras-MacBook-Air: practical_4 % ./Practical_4
Child 1: PID = 8528
Child 2: PID = 8529
Parent: PID = 8527

Using wait():
Child 3: PID = 8530
Child 2 completed.
wait() collected child PID: 8529
Exit status: 20

Using waitpid() for Child 1:
Child 3 completed.
Child 1 completed.
waitpid() collected Child 1 PID: 8528
Exit status: 10
waitpid() collected Child 3 PID: 8530
Exit status: 30
```

Task-2

Inputs and Outputs :

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child: PID = %d\n", getpid());
        printf("Child terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping...\n");

        sleep(30);

        printf("Parent terminating...\n");
    }

    return 0;
}

```

```

Parent: PID = 8550
Child PID = 8551
Parent sleeping...
Child: PID = 8551
Child terminating...
Parent terminating...
boddanapurnachandarrao@boddanas-MacBook-Air practical-4 %

```

